

1) You are given two binary strings s_1 and s_2 of equal length, and the task is to find the minimum number of swaps required to make them identical. The only allowed operation is swapping characters between the two strings (i.e., you can swap $s_1[i]$ with $s_2[j]$, but not within the same string). If it is impossible to make the two strings equal through such swaps, return -1.

The screenshot shows a coding platform interface with a problem solved successfully. The left sidebar displays the following statistics:

- Test Cases Passed:** 1120 / 1120
- Attempts:** Correct / Total: 1 / 2
- Accuracy:** 50%
- Points Scored:** 2 / 2
- Time Taken:** 0.21
- Your Total Score:** 104

The right sidebar shows the code for the `minSwaps` function in Java:

```

1 class Solution {
2     public int minSwaps(String s1, String s2) {
3         int x = 0; // count of 01 mismatches
4         int y = 0; // count of 10 mismatches
5
6         for (int i = 0; i < s1.length(); i++) {
7             char a = s1.charAt(i);
8             char b = s2.charAt(i);
9
10            if (a == b) continue;
11
12            if (a == '0' && b == '1') {
13                x++;
14            } else {
15                y++;
16            }
17        }
18
19        // Impossible if total mismatches are odd
20        if ((x + y) % 2 == 1) {
21            return -1;
22        }
23
24        // Count swaps
25        return (x / 2) + (y / 2) + (x % 2) * 2;
26    }
27 }

```

2) You are given a string s consisting only of the characters '(' and ')'. Your task is to determine the minimum number of parentheses (either '(' or ')') that must be inserted at any positions to make the string s a valid parentheses string. A parentheses string is considered valid if:

1. Every opening parenthesis '(' has a corresponding closing parenthesis ')'.
2. Every closing parenthesis ')' has a corresponding opening parenthesis '('.
3. Parentheses are properly nested.

Examples: Input: $s = "(()"$ Output: 2 Explanation: There are two unmatched '(' at the end, so we need to add two ')' to make the string valid. Input: $s = ")))"$ Output: 3 Explanation: Three '(' need to be added at the start to make the string valid. Input: $s = ")()"$ Output: 1 Explanation: The very first ')' is unmatched, so we need to add one '(' at the beginning.

The screenshot shows a coding platform interface with a problem solved successfully. The left sidebar displays the following statistics:

- Test Cases Passed:** 1111 / 1111
- Attempts:** Correct / Total: 1 / 1
- Accuracy:** 100%
- Points Scored:** 4 / 4
- Time Taken:** 0.06
- Your Total Score:** 108

The right sidebar shows the code for the `minParentheses` function in Java:

```

1 class Solution {
2     public int minParentheses(String s) {
3         int balance = 0;
4         int insertions = 0;
5
6         for (char ch : s.toCharArray()) {
7             if (ch == '(') {
8                 balance++;
9             } else {
10                if (balance > 0) {
11                    balance--;
12                } else {
13                    insertions++;
14                }
15            }
16        }
17
18        return insertions + balance;
19    }
20 }

```

3) Given a string s consisting of balanced parentheses, calculate the score of the string based on the following rules:

- $()$ has a score of 1.
- AB has a score of $A + B$, where A and B are balanced parentheses strings.
- (A) has a score of $2 \times \text{score}(A)$, where A is a balanced parentheses string.

Note: Test cases are generated such that the score will fit within a 32-bit integer. Examples: Input: $s = "()()$ Output: 2 Explanation: The string str is of the form $"ab"$, that makes the total score = (score of a) + (score of b) = $1 + 1 = 2$. Input: $s = "(()())"$ Output: 6 Explanation: The string str is of the form $"(a(b))"$ which makes the total score = $2 \times ((\text{score of } a) + 2 \times (\text{score of } b)) = 2 \times (1 + 2 \times (1)) = 6$. Input: $s = "((()))"$ Output: 4 Explanation: The string str is of the form $"((a))"$ which makes the total score = $2 \times (2 \times (\text{score of } a)) = 2 \times (2 \times (1)) = 4$.

The screenshot shows a coding platform interface with the following details:

- Problem Status:** Problem Solved Successfully (green checkmark).
- Test Cases Passed:** 1120 / 1120.
- Attempts:** Correct / Total: 1 / 1.
- Accuracy:** 100%.
- Points Scored:** 4 / 4.
- Time Taken:** 0.06.
- Your Total Score:** 112 (with a green upward arrow).
- Solve Next:** The Celebrity Problem, Get Min from Stack, Histogram Max Rectangular Area.
- Stay Ahead With:** Build 21 Projects in 21 Days (Build real-world ML, Deep Learning & Gen AI projects).
- Code Editor:** Java (21) with a solution for the parentheses problem. The code uses a stack to calculate the score based on the given rules.

4) Given an array $arr[]$ of time strings in 24-hour clock format $"HH:MM:SS"$, return the minimum difference in seconds between any two time strings in the $arr[]$. The clock wraps around at midnight, so the time difference between $"23:59:59"$ and $"00:00:00"$ is 1 second. Examples: Input: $arr[] = ["12:30:15", "12:30:45"]$ Output: 30 Explanation: The minimum time difference is 30 seconds. Input: $arr[] = ["00:00:01", "23:59:59", "00:00:05"]$ Output: 2 Explanation: The time difference is minimum between $"00:00:01"$ and $"23:59:59"$.

Problem Solved Successfully ✓

Test Cases Passed: **1120 / 1120**

Attempts: Correct / Total: **1 / 1**

Accuracy: 100%

Points Scored: **4 / 4**

Your Total Score: 116 ↑

Time Taken: **0.88**

Solve Next

Validate an IP Address Multiply two strings Implement Atoi

Stay Ahead With:

Build 21 Projects in 21 Days
Build real-world ML, Deep Learning & Gen AI projects

```

1 class Solution {
2     // Convert HH:MM:SS to total seconds
3     private int toSeconds(String time) {
4         String[] parts = time.split(":");
5
6         int h = Integer.parseInt(parts[0]);
7         int m = Integer.parseInt(parts[1]);
8         int s = Integer.parseInt(parts[2]);
9
10        return h * 3600 + m * 60 + s;
11    }
12
13    public int minDifference(String[] arr) {
14        int n = arr.length;
15        int[] seconds = new int[n];
16
17        // Convert all times into seconds
18        for (int i = 0; i < n; i++) {
19            seconds[i] = toSeconds(arr[i]);
20        }
21
22        // Sort the seconds array
23        Arrays.sort(seconds);
24
25        int minDiff = Integer.MAX_VALUE;
26
27        // Find minimum adjacent difference
28        for (int i = 1; i < n; i++) {
29            minDiff = Math.min(minDiff, seconds[i] - seconds[i - 1]);
30        }
31
32        // Check circular difference across midnight
33        int circularDiff = 86400 - seconds[n - 1] + seconds[0];
34        minDiff = Math.min(minDiff, circularDiff);
35
36        return minDiff;
37    }
38 }
39
40
41

```

5) Given a string s , the task is to arrange the string according to the frequency of each character, in ascending order. If two elements have the same frequency, then they are sorted in lexicographical order. Examples: Input: $s = \text{"geeksforgeeks"}$ Output: forggkkssee Explanation: All the characters with minimum frequency will occur first and the one with same frequency will be arranged lexicographically. Input: $s = \text{"abc"}$ Output: abc Explanation: The frequency is one for all characters hence they'll be arranged lexicographically.

Problem Solved Successfully ✓

Test Cases Passed: **1117 / 1117**

Attempts: Correct / Total: **1 / 1**

Accuracy: 100%

Points Scored: **4 / 4**

Your Total Score: 120 ↑

Time Taken: **0.31**

Solve Next

Validate an IP Address Multiply two strings Implement Atoi

Stay Ahead With:

Build 21 Projects in 21 Days
Build real-world ML, Deep Learning & Gen AI projects

```

1 class Solution {
2     public String frequencySort(String s) {
3         int[] freq = new int[26];
4
5         // Count frequency
6         for (char ch : s.toCharArray()) {
7             freq[ch - 'a']++;
8         }
9
10        // Store characters with frequency
11        List<Character> list = new ArrayList<>();
12
13        for (int i = 0; i < 26; i++) {
14            if (freq[i] > 0) {
15                list.add((char)(i + 'a'));
16            }
17        }
18
19        // Sort by frequency, then lexicographically
20        Collections.sort(list, (a, b) -> {
21            if (freq[a - 'a'] != freq[b - 'a']) {
22                return freq[a - 'a'] - freq[b - 'a'];
23            }
24            return a - b;
25        });
26
27        // Build answer
28        StringBuilder ans = new StringBuilder();
29
30        for (char ch : list) {
31            int count = freq[ch - 'a'];
32
33            while (count-- > 0) {
34                ans.append(ch);
35            }
36        }
37
38        return ans.toString();
39    }
40 }

```

6) You are given an array $\text{arr}[]$ of strings. You have to sort the array in ascending order based on the lengths of the strings. If two strings have the same length, maintain their original relative order. Examples: Input: $\text{arr}[] = [\text{"GeeksforGeeks"}, \text{"I"}, \text{"from"}, \text{"am"}]$ Output: $[\text{"I"}, \text{"am"}, \text{"from"}, \text{"GeeksforGeeks"}]$ Explanation: The strings are sorted in increasing order of their lengths, starting from the shortest string "I" to the longest one "GeeksforGeeks". Input: $\text{arr}[] = [\text{"You"}, \text{"are"}, \text{"beautiful"}, \text{"looking"}]$ Output: $[\text{"You"}, \text{"are"}, \text{"looking"}, \text{"beautiful"}]$ Explanation: The strings are

sorted by length: "You", "are", "looking", and then "beautiful", with the shortest words appearing first and the longest last.

The screenshot shows a coding platform interface. On the left, the 'Output Window' displays 'Problem Solved Successfully' with a green checkmark. It includes statistics: Test Cases Passed (1111 / 1111), Attempts: Correct / Total (1 / 1), Accuracy: 100%, Points Scored (4 / 4), and Time Taken (1.43). Below this, it suggests the next problem to solve: 'Validate an IP Address', 'Multiply two strings', and 'Implement Atoi'. On the right, the code editor shows a Java solution for sorting an array of strings by length. The code imports java.util.* and defines a class Solution with a public void sortByLength(String[] arr) method. Inside the method, Arrays.sort(arr, (a, b) -> a.length() - b.length()); is used to sort the array. The bottom of the interface has buttons for 'Custom Input', 'Compile & Run', and 'Submit'.

7) Given an array `arr[]` of strings of equal length, where each string consists of lowercase English letters, find the total number of pairs of strings that differ in exactly one character position. Note: Two strings differ in exactly one position if they have the same length and differ in exactly one index that is, all characters except one are the same. Examples : Input: `arr[] = ["abc", "abd", "bbd"]` Output: 2 Explanation: The valid pairs are: 1. ("abc", "abd") → differ at index 2 2. ("abd", "bbd") → differ at index 0 Input: `arr[] = ["def", "deg", "dmf", "xef", "dxg"]` Output: 4 Explanation: The valid pairs are: 1. ("def", "deg") → differ at index 2 2. ("def", "dmf") → differ at index 1 3. ("def", "xef") → differ at index 0 4. ("deg", "dxg") → differ at index 1 Input: `arr[] = ["bcde", "bcd", "bdce"]` Output: 0 Explanation: No pair differs at exactly one position.

The screenshot shows a coding platform interface. On the left, the 'Output Window' displays 'Problem Solved Successfully' with a green checkmark. It includes statistics: Test Cases Passed (1120 / 1120), Attempts: Correct / Total (1 / 1), Accuracy: 100%, Points Scored (4 / 4), and Time Taken (0.37). Below this, it suggests the next problem to solve: 'Validate an IP Address', 'Multiply two strings', and 'Implement Atoi'. On the right, the code editor shows a Java solution for counting pairs of strings that differ in exactly one character position. The code imports java.util.* and defines a class Solution with a public int countPairs(String[] arr) method. Inside the method, a HashMap<String, Integer> map is created. A loop iterates over each string s in arr, converting it to a char array ch. Another loop iterates over each character position i in ch, creating a pattern string by replacing ch[i] with '*'. The count of strings with the same pattern is incremented in the map. Finally, the total count of pairs is returned. The bottom of the interface has buttons for 'Custom Input', 'Compile & Run', and 'Submit'.

8) Given a lowercase string array `arr[]`. Each element in the array represents a vote cast for a candidate. Return the name of the candidate who received the maximum number of votes and the count of votes he received. In case of a tie between two or more candidates, return the lexicographically smallest name. Note: Return an array of strings, the winning candidate name as the first element and the vote count as the second element (typecast the count to string). Examples :
 Input: `arr[] = ["john", "johnny", "jackie", "johnny", "john", "jackie", "jamie", "jamie", "john", "johnny", "jamie", "johnny", "john"]` Output: `["john", "4"]` Explanation: "john" has 4 votes casted for him, but so does "johnny". "john" is lexicographically smaller, so we print "john" and the votes he received.
 Input: `n = 3 arr[] = ["andy", "blake", "clark"]` Output: `["Andy", "1"]` Explanation: All the candidates get 1 votes each. We print "andy" as it is lexicographically smaller.

The screenshot shows a coding platform interface with a problem solved successfully. The left panel displays the problem status: 'Problem Solved Successfully', 'Test Cases Passed: 1110 / 1110', 'Attempts: Correct / Total: 1 / 1', 'Accuracy: 100%', 'Points Scored: 2 / 2', and 'Time Taken: 0.28'. The right panel shows the Java code for the solution.

```

1- import java.util.*;
2
3- class Solution {
4-     public List<String> winner(List<String> arr) {
5
6         HashMap<String, Integer> map = new HashMap<>();
7
8         // Count votes
9         for (String name : arr) {
10             map.put(name, map.getOrDefault(name, 0) + 1);
11         }
12
13         String winner = "";
14         int maxVotes = 0;
15
16         // Find winner
17         for (String name : map.keySet()) {
18             int votes = map.get(name);
19
20             if (votes > maxVotes) {
21                 maxVotes = votes;
22                 winner = name;
23             }
24             else if (votes == maxVotes && name.compareTo(winner) < 0) {
25                 winner = name;
26             }
27         }
28
29         List<String> ans = new ArrayList<>();
30         ans.add(winner);
31         ans.add(String.valueOf(maxVotes));
32
33         return ans;
34     }
35 }
  
```

9) Given two strings `txt` and `pat` having lowercase letters, the task is to check if any permutation of `pat` is a substring of `txt`. Examples: Input: `txt = "geeks", pat = "eke"` Output: `true` Explanation: "eek" is a permutation of "eke" which exists in "geeks". Input: `txt = "programming", pat = "rain"` Output: `false` Explanation: No permutation of "rain" exists as a substring in "programming".

Problem Solved Successfully ✓

Test Cases Passed: **1115 / 1115**

Attempts: Correct / Total: **1 / 2**

Accuracy: 50%

Points Scored: **2 / 2**

Your Total Score: 132 ↑

Time Taken: **0.3**

Solve Next

Run Length Encoding Match specific pattern Repetition of k length substring

Stay Ahead With:

Build 21 Projects in 21 Days
Build real-world ML, Deep Learning & Gen AI projects

```

5 int m = pat.length();
6 if (m > n) return false;
7
8 int[] freq = new int[256];
9
10 // Build first window
11 for (int i = 0; i < m; i++) {
12     freq[txt.charAt(i)]++;
13     freq[pat.charAt(i)]--;
14 }
15
16 if (allZero(freq)) {
17     return true;
18 }
19
20 // Sliding window
21 for (int i = m; i < n; i++) {
22     freq[txt.charAt(i)]++;
23     freq[txt.charAt(i - m)]--;
24     if (allZero(freq)) {
25         return true;
26     }
27 }
28 return false;
29
30 private boolean allZero(int[] freq) {
31     for (int x : freq) {
32         if (x != 0) {
33             return false;
34         }
35     }
36     return true;
37 }
38
39 }
40
41
42
43
44

```

10) Given a string s consisting of lowercase English letters and an integer k , your task is to remove exactly k characters from the string. The resulting string must be the largest possible in lexicographical order, while maintain the relative order of the remaining characters. Examples: Input: $s = \text{"ritz"}$, $k = 2$ Output: "tz" Explanation: By removing two characters in all possible ways, we get: "ri" , "rt" , "rz" , "it" , "iz" , and "tz" . Among these, "tz" is lexicographically largest string. Input: $s = \text{"zebra"}$, $k = 3$ Output: "zr" Explanation: Removing "e" , "b" , and "a" results in "zr" , which is lexicographically largest string.

Problem Solved Successfully ✓

Test Cases Passed: **1120 / 1120**

Attempts: Correct / Total: **1 / 1**

Accuracy: 100%

Points Scored: **4 / 4**

Your Total Score: 136 ↑

Time Taken: **0.1**

Solve Next

Majority Element Fractional Knapsack Minimum Platforms

Stay Ahead With:

Build 21 Projects in 21 Days
Build real-world ML, Deep Learning & Gen AI projects

```

1 class Solution {
2     public String maxSubseq(String s, int k) {
3         StringBuilder ans = new StringBuilder();
4
5         for (char ch : s.toCharArray()) {
6
7             // Remove smaller previous characters
8             while (ans.length() > 0 &&
9                 ans.charAt(ans.length() - 1) < ch &&
10                 k > 0) {
11                 ans.deleteCharAt(ans.length() - 1);
12                 k--;
13             }
14             ans.append(ch);
15         }
16
17         // Remove remaining characters from end
18         while (k > 0 && ans.length() > 0) {
19             ans.deleteCharAt(ans.length() - 1);
20             k--;
21         }
22         return ans.toString();
23     }
24 }
25
26
27
28

```

11) You are given a lowercase string s , determine the total number of distinct strings that can be formed using the following rules: • Identify all unique vowels (a, e, i, o, u) present in the string. • For each distinct vowel, choose exactly one of its occurrences from s . If a vowel appears multiple times, each occurrence represents a unique selection choice. • Generate all possible permutations of the selected vowels. Each unique arrangement counts as a distinct string. Return the total number of such distinct strings. Examples: Input: $s = \text{"aeiou"}$ Output: 120 Explanation: Each vowel appears once,

so the number of different strings can form is $5! = 120$. Input: $s = "ae"$ Output: 2 Explanation: Pick a and e, make all orders $\rightarrow "ae", "ea"$. Input: $s = "aacidf"$ Output: 4 Explanation: Vowels in s are 'a' and 'i', Pick each 'a' once with a single 'i', make all orders $\rightarrow "ai", "ia", "ai", "ia"$.

The screenshot shows a coding platform interface with a problem solved successfully. The left sidebar displays the following statistics:

- Test Cases Passed: 102 / 102
- Attempts: Correct / Total: 1 / 1
- Accuracy: 100%
- Points Scored: 4 / 4
- Time Taken: 0.09
- Your Total Score: 140

The main area shows the Java code for the solution:

```

1 class Solution {
2
3     int vowelCount(String s) {
4
5         int[] freq = new int[5]; // a,e,i,o,u
6
7         // Count vowel frequencies
8         for (char ch : s.toCharArray()) {
9             if (ch == 'a') freq[0]++;
10            else if (ch == 'e') freq[1]++;
11            else if (ch == 'i') freq[2]++;
12            else if (ch == 'o') freq[3]++;
13            else if (ch == 'u') freq[4]++;
14        }
15
16        int distinct = 0;
17        int choices = 1;
18
19        // Calculate product of frequencies
20        for (int f : freq) {
21            if (f > 0) {
22                distinct++;
23                choices *= f;
24            }
25        }
26
27        // No vowels present
28        if (distinct == 0) return 0;
29
30        // Factorial of distinct vowels
31        int fact = 1;
32        for (int i = 1; i <= distinct; i++) {
33            fact *= i;
34        }
35
36        return choices * fact;
37    }
38 }

```

12) You are given a string s consisting of lowercase english letters. Your task is to count how many distinct characters appear an even number of times in the string. Examples: Input: $s = "abacaba"$ Output: 2 Explanation: The frequency of a is 4, b is 2 and c is 1 so there are 2 characters with even frequency. Input: $s = "zzacccz"$ Output: 0 Explanation: The frequency of z is 3, a is 1 and c is 3 so there are no characters with even frequency.

The screenshot shows a coding platform interface with a problem solved successfully. The left sidebar displays the following statistics:

- Test Cases Passed: 1115 / 1115
- Attempts: Correct / Total: 1 / 1
- Accuracy: 100%
- Points Scored: 2 / 2
- Time Taken: 0.28
- Your Total Score: 142

The main area shows the Java code for the solution:

```

1 class Solution {
2
3     int count(String s) {
4
5         int[] freq = new int[26];
6
7         // Count frequency of characters
8         for (char ch : s.toCharArray()) {
9             freq[ch - 'a']++;
10        }
11
12        int ans = 0;
13
14        // Count characters having even frequency
15        for (int f : freq) {
16            if (f > 0 && f % 2 == 0) {
17                ans++;
18            }
19        }
20
21        return ans;
22    }
23 }

```

13) You are given two strings, s_1 and s_2 , where s_1 contains distinct lowercase vowels (a, e, i, o, u), and s_2 contains lowercase English letters. Your task is to find the length of the shortest contiguous substring within s_2 that contains all the vowels present in s_1 at least once, in any order. Note: If there is no such substring in s_2 , return -1. Examples: Input: $s_1 = "ae"$, $s_2 = "acbaudeq"$ Output: 4

Explanation: The shortest substring of "acbaudeq" that contains both vowels 'a' and 'e' from s1 = "ae" is "aude", which has length 4. Input: s1 = "iou", s2 = "iuixoiu" Output: 3 Explanation: The shortest substring of "iuixoiu" that contains all vowels 'i', 'o', and 'u' from s1 = "iou" is "oiu", which has length 3. Input: s1 = "aeiou", s2 = "uoiee" Output: -1 Explanation: The string s2 = "uoiee" is missing the vowel 'a' from s1 = "aeiou", so no substring can contain all required vowels, and the answer is -1.

The screenshot shows a coding platform interface. On the left, the 'Output Window' displays 'Problem Solved Successfully' with a green checkmark. Below this, statistics are shown: 'Test Cases Passed: 1120 / 1120', 'Attempts: Correct / Total: 1 / 1', 'Accuracy: 100%', 'Points Scored: 4 / 4', and 'Time Taken: 0.12'. There are also buttons for 'Solve Next' and 'Stay Ahead With: Build 21 Projects in 21 Days'. On the right, a code editor shows a Java solution for a sliding window problem. The code uses a 'need' array to track required characters and a 'left' pointer to maintain a valid window. The final result is returned as -1 if no valid window is found, or the length of the minimum window otherwise.

14) You are given an array of strings arr[], where each arr[i] consists of lowercase english alphabets. You need to find the number of balanced strings in arr[] which can be formed by concatenating one or more contiguous strings of arr[]. A balanced string contains the equal number of vowels and consonants. Examples: Input: arr[] = ["aeio", "aa", "bc", "ot", "cdbc"] Output: 4 Explanation: arr[0..4], arr[1..2], arr[1..3], arr[3..3] are the balanced substrings with equal consonants and vowels. Input: arr[] = ["ab", "be"] Output: 3 Explanation: arr[0..0], arr[0..1], arr[1..1] are the balanced substrings with equal consonants and vowels. Input: arr[] = ["tz", "gfg", "ae"] Output: 0 Explanation: There is no such balanced substring present in arr[] with equal consonants and vowels.

eg

Search...

CoursesTutorialsPracticeJobs

ProblemEditorialSubmissionsComments

Output Window

Compilation ResultsCustom InputY.O.G.I. (AI Bot)

Problem Solved Successfully

Test Cases Passed
1115 / 1115

Attempts: Correct / Total
1 / 1
Accuracy: 100%

Points Scored
4 / 4
Your Total Score: 150

Time Taken
0.36

Solve Next

Validate an IP AddressMultiply two stringsImplement Atoi

Stay Ahead With:



Build 21 Projects in 21 Days
Build real-world ML, Deep Learning & Gen AI projects

Java (21)

Start Timer

```
4
5
6 private boolean isVowel(char ch) {
7     return ch == 'a' || ch == 'e' || ch == 'i'
8         || ch == 'o' || ch == 'u';
9 }
10
11 public int countBalanced(String[] arr) {
12     HashMap<Integer, Integer> map = new HashMap<>();
13
14     // Prefix sum 0 occurs once initially
15     map.put(0, 1);
16
17     int prefix = 0;
18     int ans = 0;
19
20     for (String str : arr) {
21
22         int score = 0;
23
24         // Calculate score of current string
25         for (char ch : str.toCharArray()) {
26             if (isVowel(ch)) {
27                 score++;
28             } else {
29                 score--;
30             }
31
32             prefix += score;
33
34             // Add previous occurrences
35             ans += map.getOrDefault(prefix, 0);
36             map.put(prefix, map.getOrDefault(prefix, 0) + 1);
37         }
38     }
39
40     return ans;
41 }
42
43
```

Custom InputCompile & RunSubmit